

Qt + Snap7 读取 PLC 数据 —— 小白上手指南（项目 3）

本项目和项目 2 的区别

	项目 2	项目 3
功能	连接 / 断开	连接 / 断开 + 读取数据
新增控件	—	区域选择、地址输入、HEX 显示、解析类型
新增概念	—	PLC 存储区、大端字节序、原始字节解析

PLC 有哪些"数据区"

西门子 PLC 把数据按功能分成几个区：

区域	含义	snap7 函数	例子
M 区 (Merker)	中间继电器、内部标志位	MBRead / MBWrite	M0.0 ~ M10.7
I 区 (Input)	物理输入点的映像	EBRead / EBWrite	I0.0 ~ I1.7
Q 区 (Output)	物理输出点的映像	ABRead / ABWrite	Q0.0 ~ Q1.7
DB 块 (Data Block)	用户自定义数据块	DBRead / DBWrite	DB1.DBX0.0, DB1.DBW0, DB1.DBDO

为什么 I 区叫 EB、Q 区叫 AB？ snap7 沿用了西门子德文命名：

- E = Eingang（输入）
- A = Ausgang（输出）

项目文件一览

stage3_read_data/	
├── CMakeLists.txt	← 同项目2
├── main.cpp	← 同项目2
├── mainwindow.h	← 新增读取控件声明
├── mainwindow.cpp	← 新增读取控件实现 + 数据解析逻辑
├── plc_worker.h	← 新增 readArea 槽 + dataRead 信号
├── plc_worker.cpp	← 新增 DBRead/MBRead/EBRead/ABRead 调用
└── snap7.h / snap7.cpp / snap7.dll / libsnap7_mingw.a	

核心概念：大端字节序（Big Endian）

计算机存储多字节数据有两种方式：

方式	规则	谁在用
小端 (Little Endian)	低字节放在低地址	x86 PC
大端 (Big Endian)	高字节放在低地址	西门子 PLC、网络协议

举例：16 位整数 0x1234（十进制 4660）

内存地址：	[0]	[1]	
大端存储：	12	34	← PLC 用它
小端存储：	34	12	← PC 用它

如果你把 PC 内存里的 0x1234 直接发给 PLC，PLC 会读成 0x3412（十进制 13330），完全错了。

所以读取时要“翻转”字节：在代码中我们用 getU16BE 函数把两个字节手动拼回去。

```
// 从 PLC 读到的两个字节：data[0]=0x12, data[1]=0x34
// 拼成 16 位整数：
quint16 value = ((quint8)data[0] << 8) | (quint8)data[1];
//           ↑ 高字节，左移8位           ↑ 低字节
// 结果 = 0x1234 = 4660
```

关键代码讲解

plc_worker.h 新增内容

```
// 区域代号——用 #define 比直接写数字更可读
#define AREA_M 0 // M 区
#define AREA_I 1 // I 区（snap7 的 EB）
#define AREA_Q 2 // Q 区（snap7 的 AB）
#define AREA_DB 3 // DB 块

public slots:
    void readArea(int areaType, int dbNumber, int start, int size); // 新增

signals:
    void dataRead(int areaType, int dbNumber, int start,
                  const QByteArray &data); // 新增——读到的原始字节
```

plc_worker.cpp 新增内容

```

void PlcWorker::readArea(int areaType, int dbNumber, int start, int size)
{
    QByteArray buffer(size, '\0'); // 预分配 size 字节

    // 根据区域选择不同的 snap7 函数
    switch (areaType) {
    case AREA_M: result = m_client->MBRead(start, size, buffer.data()); break;
    case AREA_I: result = m_client->EBRead(start, size, buffer.data()); break;
    case AREA_Q: result = m_client->ABRead(start, size, buffer.data()); break;
    case AREA_DB: result = m_client->DBRead(dbNumber, start, size, buffer.data()); break;
    }

    if (result == 0)
        emit dataRead(areaType, dbNumber, start, buffer);
    else
        emit errorOccurred(result, ...);
}

```

要点： `QByteArray buffer(size, '\0')` 分配一块指定大小的内存，`data()` 返回原始指针传给 snap7。

mainwindow.cpp 新增: `parseValue()`

这是项目 3 最核心的新知识。函数根据用户选择的类型，把原始字节翻译成人能读的数字：

```

QString MainWindow::parseValue(const QByteArray &data, int valueType) const
{
    switch (valueType) {
    case 0: // BOOL —— 每个字节拆成 8 个位
    case 1: // INT8 —— 直接转 signed char
    case 2: // UINT8 —— 直接转 unsigned char
    case 3: // INT16 —— 每 2 字节拼成大端整数
    case 4: // UINT16
    case 5: // INT32 —— 每 4 字节拼成大端整数
    case 6: // UINT32
    case 7: // FLOAT32 —— 每 4 字节拼成大端 float
    }
}

```

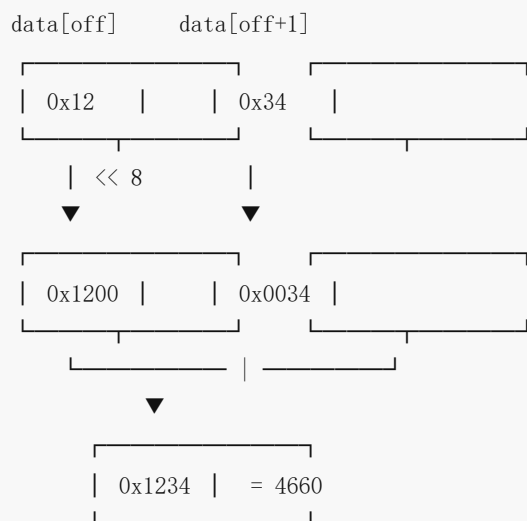
INT16 大端拼接 (最重要) :

```

auto getU16BE = [&](int off) -> quint16 {
    return ((quint8)data.at(off) << 8) | (quint8)data.at(off + 1);
};

```

<< 8 是把高字节左移 8 位，"|" 是把低字节拼上去。画成图：



mainwindow.cpp 新增: formatHex()

把原始字节格式化成 "00 0A FF 3C ..." 这种可读形式:

```
QString MainWindow::formatHex(const QByteArray &data) const
{
    QString result;
    for (int i = 0; i < data.size(); ++i) {
        if (i > 0 && i % 16 == 0) result += "\n"; // 每16字节换行
        result += QString("%1 ")
            .arg((unsigned char)data.at(i), 2, 16, QChar('0'))
            .toUpper();
    }
    return result.trimmed();
}
```

`QString("%1").arg(x, 2, 16, QChar('0'))` 的意思是: 把 x 格式化为 16 进制, 至少 2 位, 不足补 0。

UI 布局

```
| 连接设置 |
| PLC IP: [192.168.0.1] [连接] [断开] |
| ● 已连接 |
|-----|
| 读取数据 |
| 区域: [M ▼] 起始: [0] 字节数: [10] |
| 解析为: [INT16 ▼] [读取] |
| 原始数据: 00 0A FF 3C ... |
```

| 解析值: 10, -196, ... |

编译步骤

```
# 1. 配置
cmake -B build -G "Ninja" \
    -DCMAKE_CXX_COMPILER=g++ \
    -DCMAKE_PREFIX_PATH=C:/Qt/6.11.0/mingw_64

# 2. 编译
cmake --build build

# 3. 部署 DLL
cp snap7.dll build/
export PATH="/c/Qt/6.11.0/mingw_64/bin:$PATH"
cd build
windeployqt Stage3_ReadData.exe
```

常见问题

Q: 读到的全是 0

原因: PLC 对应地址确实没有值 / 地址超出范围 / 区域选错了。 **解决:** 用 TIA Portal 在线监控确认地址和值。

Q: 读 DB 块报错

原因: DB 块可能是"优化块访问"模式, snap7 只能读非优化的 DB。 **解决:** 在 TIA Portal 中, 右键 DB 块 → 属性 → 取消"优化的块访问" → 重新下载。

Q: 解析出来的值不对

原因: 选错了类型 (比如把 INT16 当 UINT16 解析, -1 变成了 65535)。 **解决:** 确认 PLC 中变量的实际类型, 选对应的解析方式。

Q: FLOAT 显示奇怪的小数

原因: 字节数不够 4 个 / 起始地址没对齐。 **解决:** float 需要 4 字节, 起始地址应该是 4 的倍数 (字对齐)。

术语速查

术语	一句话解释
M 区	PLC 内部标志位, 断电保持可选

I 区	物理输入端子状态映像
Q 区	程序计算后要输出到物理端子的值
DB 块	用户创建的数据存储区，存放任意类型的数据
大端 (Big Endian)	高字节存在低地址，西门子 PLC 使用
小端 (Little Endian)	低字节存在低地址，x86 PC 使用
字 (Word)	= 2 字节
双字 (DWord)	= 4 字节
QByteArray	Qt 的字节数组，等价于 <code>char* + size</code>